

Αναφορά για Υλοποίηση του Hash Join

Όνομα Ομάδας: dites του bounou

Μέλη Ομάδας: Παπαντωνίου Αλέξανδρος-Ιωάννης, Παπαϊωάννου Πέτρος

Αριθμοί Μητρώου μελών: 1115202200139, 1115202200137

Προδιαγραφές Μηχανήματος στο οποίο έγιναν τα πειράματα

Μοντέλο επεξεργαστή: AMD Ryzen 5 5600G 3.9 GHz

Αριθμός λογικών πυρήνων: 12

Μνήμη RAM: 16 GB

Έκδοση λειτουργικού συστήματος: Windows 10 Home-version 22H2

Τα πειράματα έγιναν σε περιβάλλον WSL Ubuntu 22.04

Παραδοτέο 1

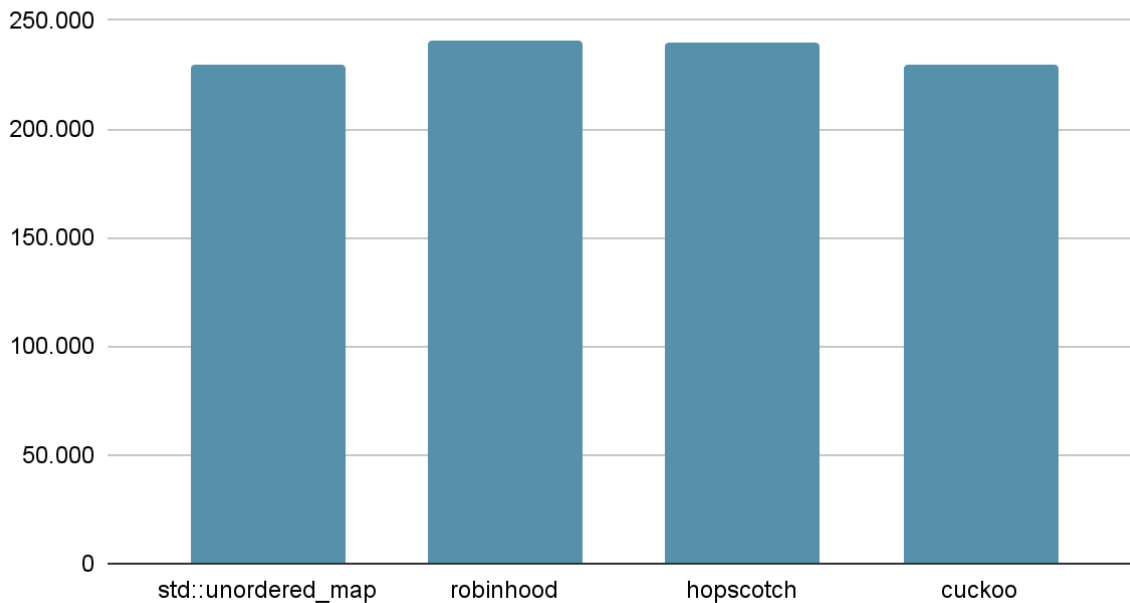
Στο παραδοτέο 1 ασχοληθήκαμε με την χρήση των διαφορετικών αλγορίθμων κατακερματισμού. Παρατηρήσαμε ότι παρ' όλες τις βελτιστοποιήσεις που προσθέσαμε δεν καταφέραμε να μειώσουμε κατά πολύ τον χρόνο εκτέλεσης σε σχέση με τον πίνακα κατακερματισμού `std::unordered_map` της c++.

Συγκεκριμένα, οι χρόνοι εκτέλεσης για κάθε αλγόριθμο φαίνονται παρακάτω:

Χρόνοι εκτέλεσης των queries

	<u>Χρόνοι εκτέλεσης των queries</u>
<code>std::unordered_map</code>	230.000 ms
<code>robinhood</code>	241.000 ms
<code>hopscotch</code>	240.000 ms
<code>cuckoo</code>	230.000 ms

Χρόνοι εκτέλεσης των queries



	Χωρίς mix hash	Με mix hash
robinhood	500.000 ms	241.000 ms
hopscotch	240.000 ms	260.000ms

Παρατηρήσεις - Βελτιστοποιήσεις

1. Οι αλγόριθμοι Robinhood και Hopscotch εκτελούν rehash όταν το load factor γίνει 0.75, ενώ ο αλγόριθμος cuckoo εκτελεί rehash όταν το load factor γίνει 0.90.
2. Αντί για την std::hash, χρησιμοποιήσαμε μία συνάρτηση mix_hash, η οποία κατανέμει καλύτερα τα στοιχεία στον πίνακα (πηγή: <https://stackoverflow.com/questions/8509180/hashing-a-small-number-to-a-random-looking-64-bit-integer>).
3. Χρήση δεύτερης emplace για το rehash η οποία δεν αντιγράφει το κλειδί και το value αλλά το μετακινεί (χρήση std::move).
4. Τροποποιήσαμε την δημιουργία του hashmap έτσι ώστε να αρχικοποιείται με μέγεθος ανάλογο του μεγέθους του left | right column αποτρέψαμε το rehashing ή το περιορίσαμε σε πολύ λίγες περιπτώσεις.
5. Όπως φαίνεται από τους χρόνους, οι διαφορετικοί αλγόριθμοι είναι περίπου το ίδιο γρήγοροι (< 5% διαφορά στον χρόνο εκτέλεσης των queries).
6. Η mix hash είναι σημαντικά πιο αποτελεσματική για τον robinhood λόγω της αποφυγής του rehashing με την διαφορετική κατανομή σε σχέση με την std::hash,

μειώνοντας έτσι τον χρόνο στο 50%. Για τον hopscotch δεν είναι τόσο αποτελεσματική, καθώς τον κάνει 8,5% πιο αργό.

Παραδοτέο 2

Ερώτημα 1

Το ερώτημα 1.1 μας δυσκόλεψε αρκετά, περισσότερο από κάθε άλλο κομμάτι της εργασίας. Αρχικά δοκιμάσαμε ένα struct το οποίο περιείχε ένα bool το οποίο υποδείκνυε αν το value_t ήταν int ή string και ένα union ενός int32 και της 64 bit αναπαράστασης για τα strings. Αυτή η υλοποίηση δεν δούλεψε (μάλλον επειδή δεν χειριζόμασταν τα nulls) και ξεπερνούσε τα 64 bits που είχαν αναφερθεί στο μάθημα.

Τελικά καταλήξαμε με την μορφή του value_t ως ένας αριθμός 64-bit ο οποίος κρατάει όλες τις απαραίτητες πληροφορίες για int32, string και null. Για να καταφέρουμε να συμπιύξουμε τις πληροφορίες σε 64 bit, παρατηρήσαμε τα μεγέθη των εισόδων στα queries και καταλήξαμε στα εξής:

1. Ο αριθμός των πινάκων δεν ξεπερνά τους 16 πίνακες (5 bit).
2. Ο αριθμός των στηλών στους πίνακες στο build side δεν ξεπερνά τις 6 (3 bit).
3. Το offset μέσα σε μία σελίδα είναι $\leq \text{PAGE_SIZE} = 8192$ (16 bit).
4. Ο αριθμός των σελίδων ανά στήλη δεν ξεπερνά τις 2^{20} (20 bit).

Άρα, συνολικά χρειαζόμαστε 44 bits για την αναπαράσταση των string και επιπλέον 3 bit για τον τύπο των δεδομένων, συνολικά 47 bit. Τα υπόλοιπα 17 bit είναι padding για να μην “χαλάσουμε” τα cache lines.

Τελικά η δομή value_t έχει την εξής μορφή:

Περιέχει έναν αριθμό 64-bit ο οποίος μπορεί να αναπαραστήσει ταυτόχρονα INT32, VARCHAR και NULL με τον παρακάτω τρόπο:

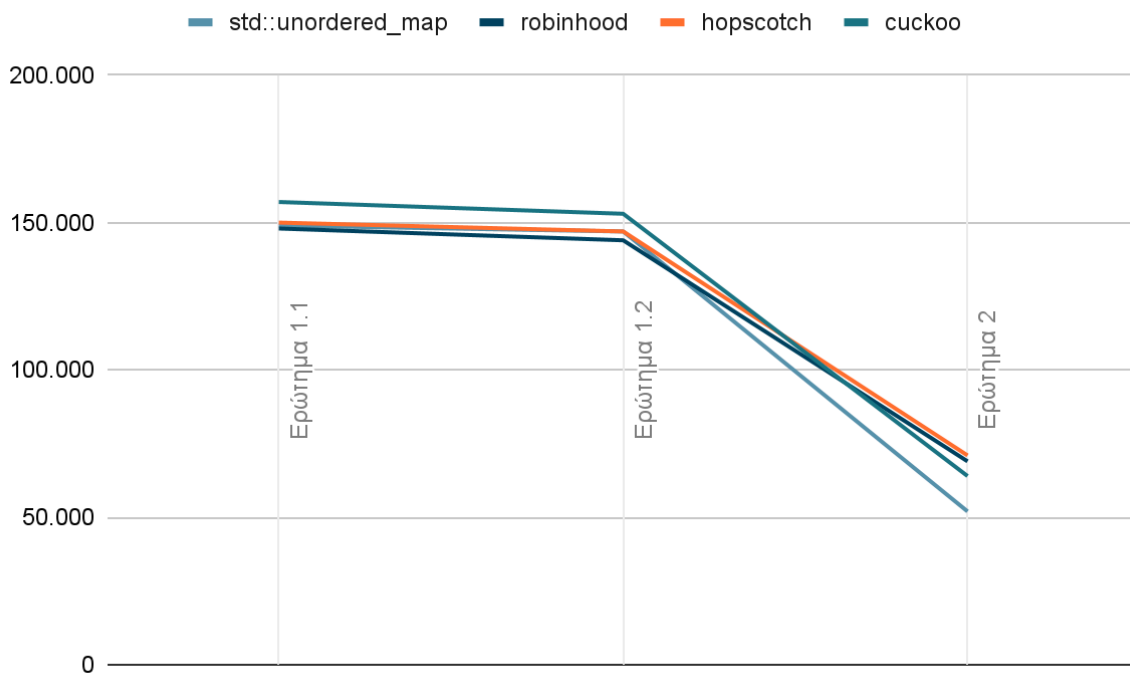
1. Τα τελευταία 3 bit (least significant) δείχνουν τον τύπο των δεδομένων που αναπαριστά.
2. Τα υπόλοιπα 61 έχουν τις πληροφορίες που χρειάζεται για την σωστή αναπαράσταση των δεδομένων:
 - a. Για INT32 κρατάμε τον αριθμό (δεν χρειάζεται να κωδικοποιήσουμε έναν απλό ακέραιο).
 - b. Για VARCHAR κρατάμε ένα string representation το οποίο περιέχει τις παρακάτω πληροφορίες:
 - i. table_id: Σε ποιον πίνακα βρίσκεται το string.
 - ii. column_id: Σε ποια στήλη του πίνακα βρίσκεται το string.
 - iii. page_id: Σε ποια σελίδα της στήλης βρίσκεται το string.
 - iv. offset: Το offset μέσα στην σελίδα που τελειώνει το string (Όπως το διαβάζουμε από τον πίνακα εισόδου).
 - c. Για NULL θέτουμε την τιμή του value_t σε 0.

Ερώτημα 2

Σε αυτό το στάδιο η δομή `column_t` λειτουργεί περίπου όπως το `Column` από το `Columnar Table`, δηλαδή αποθηκεύει τα στοιχεία σε σελίδες ανά στήλη, αλλά αντί για τις πραγματικές τους τιμές αποθηκεύει `value_t` που τα αναπαριστούν καθώς επίσης και τον αριθμό των σειρών στην στήλη. Έτσι, αποθηκεύονται τα στοιχεία σε `columnstore` μορφή κατευθείαν και δεν απαιτείται μετατροπή στο τέλος του `join`.

Χρόνοι εκτέλεσης των queries

	Ερώτημα 1.1	Ερώτημα 1.2	Ερώτημα 2
<code>std::unordered_map</code>	149.000 ms	147.000 ms	52.000 ms
<code>robinhood</code>	148.000 ms	144.000 ms	69.000 ms
<code>hopscotch</code>	150.000 ms	147.000ms	71.000 ms
<code>cuckoo</code>	157.000 ms	153.000ms	64.000 ms



Παρατηρήσεις

1. Η μεγαλύτερη βελτίωση συμβαίνει με την χρήση του value_t (αποφυγή αντιγραφών string), περίπου 40 - 45% βελτίωση στον χρόνο εκτέλεσης (ανάλογα με τον πίνακα κατακερματισμού) αλλά και βελτίωση στην μνήμη που απαιτείται.
2. Από την βελτίωση σε απευθείας κατασκευή columnstore δομής (Ερώτημα 1.2), ο χρόνος εκτέλεσης δεν μειώνεται δραματικά, περίπου 15% βελτίωση στον χρόνο εκτέλεσης.
3. Μεγάλο άλμα στην βελτίωση του χρόνου παρατηρούμε και από την εισαγωγή του column_t στην υλοποίηση, το οποίο διαχειρίζεται καλύτερα τα value_t και μειώνει τον χρόνο εκτέλεσης κατά 50 - 60% ανάλογα με τον πίνακα κατακερματισμού.

Ερώτημα 3

Δομή unchained hash table:

1. Directory: Αποτελείται από directory entries, οι οποίοι είναι αριθμοί 64-bit και αποτελούνται από έναν 48-bit 'δείκτη' (index) που δείχνει σε ποια θέση του πίνακα με τα ζευγάρια key-value ξεκινάει η δεικτιοδότηση του συγκεκριμένου slot και από ένα 16-bit φίλτρο (bloom filter) το οποίο χρησιμοποιείται για την γρήγορη απόρριψη κλειδιών κατά την αναζήτηση τα οποία δεν μπορεί να υπάρχουν στο hash table.
2. Tuples: Ένας πίνακας με τα ζευγάρια key-value ταξινομημένα με βάση το hash των κλειδιών.

Επειδή η αλλαγή του directory είναι χρονοβόρα (query 2a: 47000ms), δεν γίνεται σε κάθε προσθήκη στοιχείου στον πίνακα. Αντίθετα, όταν προσθέτουμε ένα καινούργιο στοιχείο δεν αλλάζουμε το directory, απλά το εισάγονται στο τέλος του πίνακα με τα tuples. Όταν έχουμε προσθέσει όλα τα στοιχεία που θέλουμε (κατά το build phase του join algorithm), καλούμε την create_directory() η οποία ταξινομεί τα tuples και δημιουργεί τα directory entries για να χρησιμοποιηθούν στην αναζήτηση (κατά το probe phase του join algorithm). Τυχόν διπλότυπα κλειδιά εισάγονται πολλές φορές στον πίνακα.

Η αναζήτηση στο unchained hashmap γίνεται στα εξής δύο βήματα από τον χρήστη:

1. Αρχικά καλεί την συνάρτηση lookup_range(key) η οποία επιστρέφει τις θέσεις στον πίνακα που αντιστοιχούν στο κλειδί που ψάχνει.
2. Στην συνέχεια χρησιμοποιεί αυτό το range για να ψάξει τις ανάμεσα θέσεις για το κλειδί.

Χρόνοι εκτέλεσης των queries

	unchained hashtable
hardware accelerated crc32	56.000 ms
fibonacci	50.000 ms

Προβολή στο github repository

Στο github έχουμε branches σχετικά με τα ζητούμενα της δεύτερης εργασίας. Συγκεκριμένα, στο branch askisi1_1 είναι τα αρχεία όπως τα είχαμε για το 1_1 ζητούμενο της εργασίας 2. Στο branch askisi_1_2 έχουν ενημερωθεί τα αρχεία ώστε να τρέχουν σύμφωνα με το ζητούμενο 1_2. Όμοια στο branch askisi 2 για το ζητούμενο 2 και στο branch unchained για το ζητούμενο 3. Αξίζει να σημειωθεί ότι σε όλα τα branches έχουμε την τελευταία για την εργασία 2 έκδοση των test των σχετικών αρχείων, παρόλο που για παράδειγμα στο ζητούμενο 1_1 δεν χρησιμοποιούμε το column_t.

Παραδοτέο 3

Σχεδιαστικές επιλογές

1. Slab Allocator

- a. Level 1 Slab: Δεσμεύει μνήμη σε μεγάλα chunks η οποία θα παραχωρηθεί στα threads.
- b. Level 2 Slab: Χωρίζει την μνήμη που έχει δεσμεύσει ο level 1 slab allocator σε μικρότερα chunks, δεικτοδοτεί την αρχή του και την αποδίδει σε ένα partition.
- c. Level 3 Slab: Γεμίζει τα μικρά chunks που παίρνει από τον level 2 slab allocator με τα tuples.
- d. Collector: Κάθε thread έχει έναν collector, του οποίου γεμίζει τους level 3 slab allocators (1 για κάθε partition).

2. Βελτιστοποίηση indexing

- a. Προσθέσαμε 2 νέα πεδία στην δομή column_t:
 - i. bool has_nulls: Δείχνει αν η στήλη που αναπαριστά έχει null στοιχεία.
 - ii. const Column* orig_col: Δείκτης στην στήλη του input πίνακα.
- b. Η επεξεργασία του πίνακα τώρα στην my_copv αλλάζει και γίνεται ως εξής:
 - i. Για κάθε στήλη ελέγχουμε αν υπάρχει έστω και μία σελίδα με null στοιχεία.
 - ii. Αν δεν βρούμε σελίδα με null στοιχεία, σημαίνει ότι δεν χρειάζεται να δημιουργήσουμε στήλη από value_t, οπότε αποθηκεύουμε έναν δείκτη στην στήλη του αρχικού πίνακα.
 - iii. Αν βρούμε έστω και μία σελίδα με null στοιχεία, μετατρέπουμε την στήλη σε value_t και την αποθηκεύουμε στο column_t.
- c. Η ανάκτηση των δεδομένων στην get_value άλλαξε και γίνεται ως εξής:
 - i. Αν η στήλη έχει null στοιχεία (πεδίο has_nulls == true), τότε παίρνουμε την τιμή από τις σελίδες που δημιουργήσαμε στο column_t με τα value_t.
 - ii. Αν δεν έχει null στοιχεία, τότε παίρνουμε την τιμή από την στήλη που έχουμε αποθηκεύσει στο column_t.
 - iii. Και οι δύο περιπτώσεις έχουν πολυπλοκότητα $O(1)$ αφού μέσω του αριθμού σειράς μπορούμε να βρούμε κατευθείαν σε ποιο σημείο βρίσκεται το στοιχείο (σελίδα και offset μέσα στην σελίδα).

3. Παραλληλοποίηση

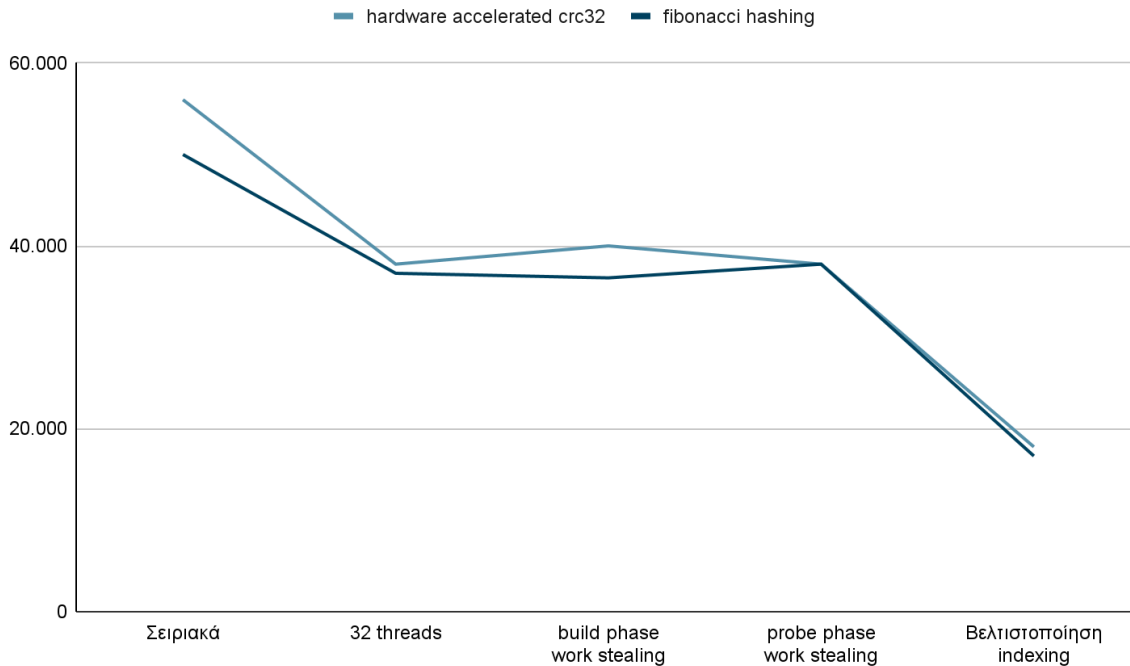
- a. Παραλληλοποίηση build phase: Κάθε worker thread αναλαμβάνει $\text{build_rows} / \text{NUM_PARTITIONS}$ σειρές του πίνακα.
- b. Παραλληλοποίηση merge phase: Κάθε worker thread αναλαμβάνει ένα partition και ακολουθεί τα εξής 3 βήματα:
 - i. Υπολογίζει τον αριθμό των tuples που αντιστοιχούν στο partition και φτιάχνει τα bloom filters του directory.
 - ii. Υπολογίζει τους δείκτες του directory (που ξεκινάει το κάθε partition).
 - iii. Αντιγράφει τα tuples στις σωστές θέσεις στο hash map.
- c. Παραλληλοποίηση probe phase: Κάθε worker thread αναλαμβάνει $\text{build_rows} / \text{NUM_PARTITIONS}$ σειρές.
- d. Παραλληλοποίηση materialize: Κάθε worker thread αναλαμβάνει $\text{num_columns} / \text{NUM_PARTITIONS}$.

4. Work Stealing

- a. Build phase: Για να κάνουμε πιο αποδοτικό το work stealing στο build phase, “αφαιρέσαμε” την λογική ότι “κάθε thread αναλαμβάνει συγκεκριμένες σειρές”. Αντίθετα, αφήσαμε κάθε thread να μπορεί να επεξεργαστεί όλες τις σελίδες του column_t . Συγκεκριμένα, έχουμε έναν global atomic μετρητή που δείχνει μέχρι ποια σειρά έχει ολοκληρωθεί το build (αυξάνουμε κάθε φορά κατά page rows) ώστε το κάθε thread που τελειώνει με μία σελίδα μπορεί να αναλάβει την επόμενη διαθέσιμη.
- b. Probe phase: Όμοια με το build phase.

Χρόνοι εκτέλεσης των queries

	Σειριακά	32 threads	build phase work stealing	probe phase work stealing	Βελτιστοποίηση indexing
hardware accelerated crc32	56.000 ms	38.000 ms	40.000 ms	38.000 ms	18.000 ms
fibonacci hashing	50.000 ms	37.000 ms	36.500 ms	38.000 ms	17.000 ms



Παρατηρήσεις

Αρχικά να αναφέρουμε ότι σε κάθε στήλη του πίνακα, ισχύουν και οι παράμετροι των προηγούμενων στηλών (π.χ. ο χρόνος που αναγράφεται για το probe phase work stealing περιέχει σαν παραμέτρους 32 threads και build phase work stealing).

1. Η εισαγωγή του παράλληλου αλγορίθμου μειώνει τον χρόνο εκτέλεσης κατά περίπου 20-30%.
2. Η μεγαλύτερη βελτίωση στην απόδοση προκαλείται από την εισαγωγή του βελτιστοποιημένου indexing, που έχει ως αποτέλεσμα την μείωση της εκτέλεσης κατά περίπου 55-60% σε σχέση με τον παράλληλο αλγόριθμο με work stealing.

Παρατηρήσεις/διαφορετικές υλοποιήσεις

1. Ο παράλληλος αλγόριθμος τρέχει πιο γρήγορα με 128KB small chunk size και large chunk size = 32 small chunk size * NUM PARTITIONS με min = 4MB και threads = NUM PARTITIONS = 32. Χρόνος 41.000 ms πριν την βελτιστοποίηση του indexing.
2. Στο probe phase είχαμε δοκιμάσει μια υλοποίηση στην οποία αντί να φτιάχνουν τα threads Execute Result να φτιάχνουν vector από tuples <size_t c (column), value_t > αλλά αυτή η υλοποίηση ήταν πιο αργή.

Βελτιστοποιήσεις στο παραδοτέο 3 για το παραδοτέο 2 μετά την προφορική

1. Προσθέσαμε test για τα long strings
2. Προσθέσαμε την συνάρτηση κατακερματισμού fibonacci για να συγκρίνουμε την επίδοση του με τον crc32